Course: Enterprise Real-Time Data Engineering
Instructor: Mohamed Fouad Fakhruldeen (Fakhr.me)
Course Portal: fakhr.me/KafkaSpark
Code & Labs Repository: github.com/fakhruldeen/KafkaSpark

Session S02: Hands-On Lab Walkthrough

Topic Creation, Partition Key Routing, and Live Broker Failover Testing

Lab Objectives

By the end of this lab, you will:
1. Provision a distributed topic with multiple partitions ($P = 3$) and full replication ($R = 3$).
2. Analyze the metadata distribution: Leader, Replica assignment, and In-Sync Replica (ISR) status.
3. Stream messages with partition keys and mathematically verify how MurmurHash2 guarantees partition stickiness for identical keys.
4. Execute an intentional Chaos Engineering test: kill an active partition Leader broker (`docker stop`) and verify zero message loss with seamless follower promotion.
5. Restore the failed broker and observe automatic ISR re-synchronization.

Step 1: Pre-Flight Verification

Verify that the 3-broker KRaft cluster is up and operational.
From your host terminal in `00_Lab` :

```
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}" | grep kafka
```

Expected output:

```
kafka1   Up ...   0.0.0.0:9092→9094/tcp
kafka2   Up ...   0.0.0.0:9093→9094/tcp
kafka3   Up ...   0.0.0.0:9094→9094/tcp
```

(Notice: External host ports 9092, 9093, 9094 forward traffic to listener port 9094 inside each container)

Step 2: Create a Fault-Tolerant Distributed Topic

Create a topic named `nile.inventory-events` with **3 partitions** and a **replication factor of 3**:

```
docker exec -it kafka1 /opt/kafka/bin/kafka-topics.sh --bootstrap-server kafka1:9092 \
  --create \
  --topic nile.inventory-events \
  --partitions 3 \
  --replication-factor 3 \
  --config min.insync.replicas=2
```

Step 3: Inspect Physical Partition Layout & Metadata

Run `--describe` to inspect how Kafka distributed partitions across the 3 brokers:

```
docker exec -it kafka1 /opt/kafka/bin/kafka-topics.sh --bootstrap-server kafka1:9092 \
  --describe \
  --topic nile.inventory-events
```

Analyze the Output:

```
Topic: nile.inventory-events    TopicId: ...    PartitionCount: 3   ReplicationFactor: 3
Configs: min.insync.replicas=2
    Topic: nile.inventory-events    Partition: 0    Leader: 1   Replicas: 1,2,3 Isr: 1,2,3
    Topic: nile.inventory-events    Partition: 1    Leader: 2   Replicas: 2,3,1 Isr: 2,3,1
    Topic: nile.inventory-events    Partition: 2    Leader: 3   Replicas: 3,1,2 Isr: 3,1,2
```

Notice:

- Each partition has a different **Leader** to distribute write workload evenly.
- All 3 brokers are in the **Isr** list (`1,2,3`).

Step 4: Verify Partition Key Hashing (Deterministic Routing)

Now open a terminal to consume messages from all partitions, printing the partition number, key, and value:

```
docker exec -it kafka1 /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server kafka1:90
92 \
  --topic nile.inventory-events \
  --from-beginning \
  --property print.key=true \
  --property print.partition=true \
  --property print.offset=true
```

In a second terminal, launch the console producer configured for key-value pairs (colon separated):

```
docker exec -it kafka1 /opt/kafka/bin/kafka-console-producer.sh --bootstrap-server kafka1:90
92 \
  --topic nile.inventory-events \
  --property "parse.key=true" \
  --property "key.separator=:"
```

Send the following inventory updates:

```
SKU-APPLE-15:{"item":"iPhone 15 Pro", "delta": -1}
SKU-SAMS-S24:{"item":"Galaxy S24", "delta": -2}
SKU-APPLE-15:{"item":"iPhone 15 Pro", "delta": +50}
SKU-DELL-XPS:{"item":"Dell XPS 15", "delta": -1}
SKU-APPLE-15:{"item":"iPhone 15 Pro", "delta": -3}
```

Observation in Consumer Terminal:

Observe that all records with key `SKU-APPLE-15` landed on the **exact same partition** (e.g., Partition 1) in strict sequential offset order (e.g., offsets 0, 1, 2). This guarantees that inventory counts for `SKU-APPLE-15` will never suffer race conditions or out-of-order decrements.

Step 5: Live Chaos Engineering - Kill a Leader Broker

Identify which broker is the Leader for Partition 1 (from Step 3, let's assume it is `kafka2`).
In your host terminal, simulate a hardware failure or kernel panic by stopping that broker:

```
docker stop kafka2
```

Immediately re-describe the topic from `kafka1`:

```
docker exec -it kafka1 /opt/kafka/bin/kafka-topics.sh --bootstrap-server kafka1:9092 \
  --describe \
  --topic nile.inventory-events
```

Expected Result:

```
Topic: nile.inventory-events    Partition: 1    Leader: 3   Replicas: 2,3,1 Isr: 3,1
```

- **Leader for Partition 1** automatically moved from `2` to `3` (or `1`) in milliseconds without human intervention.
- Broker `2` was removed from the **Isr** list.
- **The cluster remains completely operational** because `min.insync.replicas=2` is satisfied by the remaining 2 healthy brokers.

Step 6: Produce During Broker Outage

Produce another message with key `SKU-APPLE-15`:

```
SKU-APPLE-15:{"item":"iPhone 15 Pro", "delta": -1, "status": "post-crash"}
```

Notice the message is accepted instantly. The producer never crashed.

Step 7: Broker Recovery and Re-Synchronization

Restart the failed broker:

```
docker start kafka2
sleep 5
```

Re-describe the topic once more:

```
docker exec -it kafka1 /opt/kafka/bin/kafka-topics.sh --bootstrap-server kafka1:9092 \
  --describe \
  --topic nile.inventory-events
```

Expected Result:

Broker `2` caught up with all committed offsets and rejoined the ISR list: `Isr: 3,1,2` !

Step 8: Clean Up

```
docker exec -it kafka1 /opt/kafka/bin/kafka-topics.sh --bootstrap-server kafka1:9092 \
  --delete --topic nile.inventory-events
```

Congratulations! You have verified distributed partition routing and high-availability leader failover.

🌟 Gold Stretch Challenge: Log Compaction Verification

Distinguished Engineer Challenge: If you complete the core lab before time expires, execute this log compaction lifecycle verification:

1. Create a specialized compacted topic with aggressive segment rolling settings:

```bash
docker exec -it kafka1 /opt/kafka/bin/kafka-topics.sh --bootstrap-server kafka1:9092 --create --topic retail.user-profiles --partitions 3 --replication-factor 3 --config cleanup.policy=compact --config min.cleanable.dirty.ratio=0.01 --config segment.ms=5000
```

2. Publish successive state updates for the same entity key (`USER_001`) with different attribute values.
3. Wait for the log cleaner thread to execute a compaction cycle, then verify with `kafka-console-consumer.sh` that only the latest state is retained per key.